

Future Token Prediction in Small GPT-Style Models: Auxiliary Heads, Myopia, and Multi-Token Training

Harley Trae Gribble

2025-12-15

Abstract

Next-token prediction forces transformer language models to compress future information into hidden states that are reused at later time steps. Recent work shows that these hidden states contain signals about future tokens, but disagrees on whether this reflects deliberate “planning ahead” or incidental “breadcrumbs.” At the same time, multi-token prediction (MTP) architectures introduce parallel output heads trained to predict several future tokens at once, improving efficiency and, in some regimes, accuracy. In this independent study, I investigate how much future-token information small GPT-style models can exploit when the transformer “trunk” is frozen and only additional output heads are trained. I implement a decoder-only transformer from scratch in NumPy, reimplement a PyTorch GPT-style decoder with modular heads, attach auxiliary future-token heads to GPT-2 small, and train a Meta-style multi-token model from scratch on FineWeb. I evaluate cross-entropy loss, top-k accuracy, qualitative predictions, and McNemar significance tests for offsets $t+1$, $t+2$, and $t+3$. Across experiments, auxiliary heads substantially improve $t+1$ accuracy when initialized from the pretrained head but fail to achieve strong direct $t+2$ prediction, and a from-scratch two-head MTP model learns a modest $t+2$ predictor that remains much weaker than $t+1$. These results support breadcrumbs-style explanations and suggest that the benefits reported in larger MTP systems depend critically on trunk training and scale rather than auxiliary heads alone.

1 Introduction

Autoregressive language models are trained to predict the next token given a prefix. At time step t , the model receives tokens $x_1 \dots x_t$ and outputs a distribution over x_{t+1} . Despite this per-token objective, prior work shows that hidden states often contain information about tokens

several steps into the future. This raises a natural question: **do small GPT-style models “plan ahead” for future tokens, and can we train explicit heads to exploit that information?**

Wu et al. (2024) distinguish two possibilities. Under **pre-caching**, a transformer deliberately computes features at time t that are not needed for x_{t+1} but will be useful later. Under **breadcrumbs**, features that best support x_{t+1} also happen to be useful at x_{t+2} , so hidden states are predictive of the future without explicit planning. They introduce **myopic training**, which removes gradient signals from future positions, and show that small models (such as GPT-2) have a small “myopia gap,” while larger models show increasing evidence of pre-caching.

In parallel, Meta’s multi-token prediction (MTP) work replaces a single next-token head with multiple heads that each predict $x_{t+1} \dots x_{t+n}$ from the same trunk representation. Their results show substantial gains in code and byte-level modeling, along with $3\text{--}6\times$ inference speedups, but only for sufficiently large models and end-to-end MTP training.

This project investigates **whether similar benefits appear in small GPT-style models when the trunk is frozen and only additional heads are trained**. Concretely, I ask:

1. How much future-token predictability can auxiliary heads recover from the hidden states of GPT-2 small when the trunk is frozen?
2. Does reusing the pretrained next-token head as an initialization for a new $t+1$ head help future-token heads ($t+2$, $t+3$) learn stronger signals?
3. How does a from-scratch small MTP-style model behave on natural text when jointly trained to predict both $t+1$ and $t+2$?
4. How do these empirical results relate to myopia vs. pre-caching explanations and to the benefits of full MTP training reported by Meta?

My main contributions are:

- A **NumPy implementation** of a decoder-only transformer and a toy multi-head character-level MTP model, used as a controlled environment to test the mechanics of multi-token prediction.
- A **PyTorch GPT-style decoder** with modular heads, enabling frozen-trunk experiments on GPT-2 small with additional $t+1$, $t+2$, and $t+3$ heads.
- Two main **frozen-trunk GPT-2 experiments**: (i) purely random multi-head training and (ii) a **cloned $t+1$ head plus random $t+2$ head**. These experiments quantify how much auxiliary heads can improve or degrade future-token prediction.
- A **from-scratch Meta-style MTP model** (“FutureGPT”) trained on streaming FineWeb with two heads ($t+1$, $t+2$), evaluated on 800k tokens with loss, perplexity, and top-k accuracy.
- Qualitative analyses of predictions and small side experiments on representation geometry (UMAP and similarity heatmaps) and “innate concepts” in LLaMA-2 embeddings, situating the findings within Wu et al. and Meta’s results.

Overall, the results show that auxiliary heads on a frozen GPT-2 trunk can modestly exploit future-token structure but do not reach strong pre-caching-like behavior, reinforcing the view that trunk training and scale are essential for Meta-style MTP gains.

2 Background / Theory

2.1 Decoder-only Transformers

A decoder-only transformer maps an input sequence of token IDs to a sequence of hidden states via stacked self-attention and feed-forward blocks. After embedding tokens and adding positional encodings, each layer applies:

- **Causal self-attention**, where each position attends only to positions $\leq t$ using a mask.
- A **feed-forward network** applied independently to each position.
- **Residual connections** and layer normalization.

The final hidden state h_t at each position is fed into a linear output head to produce logits over the vocabulary, and softmax gives $P(x_{t+1} \mid x_1 \dots x_t)$.

2.2 Next-token training and gradients

Standard training minimizes the average cross-entropy loss over all positions:

$$L = - \sum_t \log P(x_{t+1} \mid x_1 \dots x_t).$$

Gradients backpropagate through all layers and across positions: the parameters that contribute to the representation at position t receive gradients not only from the loss at t but also from losses at later positions. Wu et al. show that these **off-diagonal gradient terms** are exactly the training signals that encourage pre-caching of features at earlier positions for future use.

2.3 Myopia, breadcrumbs, and pre-caching

Wu et al. define a **myopic transformer** as one whose parameters minimize, in aggregate, the loss at each position conditioned on a fixed past, explicitly removing off-diagonal gradient contributions from future positions. The **myopia gap** is the performance difference between a standard transformer and a myopic one trained on the same data. A small gap suggests that the model mainly relies on **breadcrumbs**—features optimized for the immediate token that incidentally help the future—while a large gap indicates pre-caching.

For small GPT-2 models, Wu et al. find a small myopia gap, consistent with breadcrumbs. At larger scales (e.g., Pythia 2.8B), they observe a growing gap, indicating more deliberate pre-caching.

2.4 Multi-token prediction and auxiliary heads

Meta’s multi-token prediction framework adds multiple output heads to a shared trunk, each predicting a different future token from the same trunk representation. During training, the loss sums over offsets (e.g., $t+1\dots t+4$), and the trunk receives gradient signals from all heads. At inference, a single head can be used for standard next-token decoding, or multiple heads can drive speculative decoding.

In this work, I explore a related but more constrained setup:

- Keep the GPT-2 trunk **frozen**.
- Remove its original LM head.
- Attach new linear heads for $t+1$, $t+2$, and $t+3$.
- Train only these heads under cross-entropy, leaving the trunk untouched.

This isolates how much future-token structure already lives in the frozen hidden states and how far auxiliary heads alone can push performance.

3 Related Work

3.1 Do Language Models Plan Ahead for Future Tokens? (Wu et al., 2024)

Wu et al. investigate whether transformers plan ahead via pre-caching or simply leave breadcrumbs. They introduce **myopic training** that removes gradient signals from future positions, making pre-caching impossible while preserving breadcrumbs. On a synthetic task where success requires explicit pre-caching (precomputing $\sin(bx_n)$ at earlier positions), standard transformers learn to pre-cache, while myopic ones cannot. On natural language modeling, small GPT-2 models show a small myopia gap, suggesting mostly breadcrumbs, whereas larger models start to rely more on pre-caching. Their work provides the conceptual lens for interpreting my auxiliary-head experiments.

3.2 Better & Faster LLMs via Multi-Token Prediction (Meta, 2024)

Meta proposes training decoder-only LLMs with **multiple output heads** that each predict a different future token from the same trunk representation. For example, a 4-head model predicts $x_{t+1} \dots x_{t+4}$ at each position. They show that, for sufficiently large models, MTP

improves performance on code and byte-level tasks and enables substantial speedups via built-in speculative decoding. However, they also report that MTP does not uniformly help all tasks and may slightly hurt small models or some language benchmarks. My from-scratch small MTP model and frozen-head experiments can be viewed as probing the low-scale regime where Meta’s benefits are weakest.

4 Methodology

4.1 NumPy Transformer and Toy Multi-Token Model

To build intuition, I implemented a small decoder-only transformer in NumPy:

- Token and positional embeddings combined into $H^{(0)}$.
- Multi-head causal attention implemented with explicit matrix operations and a top-triangular mask.
- Feed-forward layers with ReLU and residual connections.
- A simple training loop using gradient descent and manually computed cross-entropy.

On top of this model, I built a toy **TinyGPT_MTP** character-level model for a repetitive dataset (“The cat likes to nap. The cat likes to eat.”, repeated many times). TinyGPT_MTP attached three heads predicting offsets $t+1$, $t+2$, and $t+3$. Training logs showed loss decreasing over a few hundred steps, and sampled generations produced plausible but imperfect character sequences. This served as a sanity check that the multi-head architecture and masking behaved as expected.

4.2 PyTorch GPT-style Decoder

I reimplemented a GPT-style decoder block in PyTorch, mirroring the structure of GPT-2:

- Learned token embeddings and sinusoidal or learned positional embeddings.
- Multi-head self-attention with causal masking.
- Feed-forward network with GELU and residuals.
- LayerNorm in a pre-norm configuration.
- A modular output layer that can host one or more linear heads.

This implementation served both toy experiments and as a scaffold for attaching auxiliary heads to the actual GPT-2 small model from HuggingFace.

4.3 GPT-2 Auxiliary Head Architecture

For the main GPT-2 experiments, I:

- Loaded **GPT-2 small** (117M parameters) from HuggingFace.
- Removed its original language modeling head.
- Froze all trunk parameters.
- Attached new linear heads that map the final hidden state h_t to logits for:
 - **t+1** (next token),
 - **t+2** (two steps ahead),
 - optionally **t+3**.

Two variants were studied:

1. Random-heads variant (Experiment 1)

- All heads (t+1, t+2) initialized randomly from a standard normal scaled similarly to GPT-2’s default initialization.
- Trained jointly on t+1 and t+2 cross-entropy.

2. Cloned t+1 head + random t+2 head (Experiment 2)

- The t+1 head initialized by copying GPT-2’s original LM head weights.
- The t+2 head initialized randomly.
- Trained jointly on both offsets while the trunk remains frozen.

In both cases, the loss at each position is the sum of cross-entropies for the available offsets, and gradients update only the auxiliary heads.

4.4 Dataset and Tokenization

For GPT-2 experiments, I used **WikiText-2** loaded via HuggingFace **datasets**:

- Tokenization with the GPT-2 BPE tokenizer.
- Typical sequence length around 128–256 tokens.
- Teacher-forcing evaluation: given a prefix, predict gold tokens at offsets $k = +1, +2, +3$ without feeding in model predictions.

For the from-scratch Meta-style experiment, I used a **streaming sample from FineWeb**. For evaluation, I processed a streaming subset of 819,200 tokens.

4.5 Training Setup

For GPT-2 auxiliary-head experiments:

- Optimizer: Adam with a small learning rate (on the order of 10^{-10}).
- Trunk frozen; only head parameters updated.
- Training over a few epochs with periodic logging:
 - Combined training loss.
 - Per-offset hits@1 on the training subset.

For the from-scratch FutureGPT model:

- Architecture: smaller GPT-style decoder trained from scratch, with two heads for $t+1$ and $t+2$.
- Loss: sum of cross-entropies for $t+1$ and $t+2$.
- Training schedule with changing loss weights λ_1, λ_2 over epochs as documented in the notebook:
 - Initial epochs with $\lambda_1 = 1.0, \lambda_2 = 1.0$ at learning rate 5×10^{-5} .
 - Later epochs reducing λ_1 while keeping $\lambda_2 = 1.0$ and halving the learning rate, to keep $t+2$ sufficiently weighted while stabilizing $t+1$.

4.6 Evaluation Metrics

For GPT-2 and auxiliary heads, I computed:

- **Cross-entropy loss** for each offset k .
- **Top-k accuracy (hits@k)** for $k \in \{1, 3, 5\}$ where available.
- For comparison against Wu-style framing, I treated correctness of the top-1 prediction as a binary variable and applied **McNemar’s exact test** to paired predictions (vanilla vs. auxiliary heads).

For the from-scratch FutureGPT model ($t+1$ and $t+2$ heads), I computed on 819,200 FineWeb tokens:

- Average loss and perplexity per offset.
- Top-1 and top-5 accuracies.
- Qualitative examples showing context, gold tokens at $t+1$ and $t+2$, and the top predictions from each head.

5 Experiments

5.1 Baseline GPT-2 (teacher-forced)

As a baseline, I evaluated vanilla GPT-2 small on WikiText-2 using teacher forcing with offsets $k = +1$ and $k = +2$. For evaluation, I used tens of thousands of token positions per offset.

Baseline results (schematically):

Offset k	hits@1	hits@3	hits@5	Loss
+1	~0.33	~0.48	~0.54	~3.9
+2	~0.33	~0.48	~0.55	~3.8

These values confirm that GPT-2 small has a fairly strong signal even at $k = +2$ under teacher forcing: the top-1 accuracy at +2 is comparable to +1 on this dataset.

5.2 Experiment 1 – Random Auxiliary Heads (Frozen Trunk)

In the first GPT-2 auxiliary-head experiment, I:

- Removed the original LM head.
- Attached two randomly initialized linear heads for $k = +1$ and $k = +2$.
- Froze the trunk entirely.
- Trained the heads jointly on the sum of cross-entropies for $k = +1$ and $k = +2$.

Training logs showed:

- Initial steps: high combined train loss with low hits@1 (around the random baseline).
- After a couple of epochs: train loss dropped substantially, and hits@1 increased for both offsets, but remained below the vanilla GPT-2 baseline.

Final evaluation on the dev set, compared qualitatively to baseline:

- $k = +1$: auxiliary hits@1 somewhat below vanilla, with higher loss.
- $k = +2$: auxiliary hits@1 well below vanilla, with significantly higher loss.

McNemar’s tests comparing correctness of the top-1 predictions on paired positions indicated that these differences are statistically significant. Qualitatively, these results show that with a frozen trunk and purely random heads, **auxiliary heads fail to recover baseline performance**, especially at offset +2.

5.3 Experiment 2 – Cloned $t+1$ Head + Random $t+2$ Head

In the second GPT-2 experiment, I reused more information from the original model:

- The $t+1$ head was **initialized by cloning the original GPT-2 LM head**.
- The $t+2$ head was initialized randomly.
- The trunk remained frozen, and both heads were trained jointly.

During training:

- Train loss per batch decreased steadily over two epochs.
- Training hits@1 for the aux $+1$ head climbed to slightly above the vanilla baseline on the training subset, while the $+2$ head reached a modest hits@1.

Final evaluation on the dev set:

- $k = +1$: the cloned aux head **slightly outperformed** the original GPT-2 head in hits@1, with a comparable loss.
- $k = +2$: the aux head still underperformed vanilla by a large margin, both in accuracy and loss.

McNemar’s exact test for top-1 correctness showed:

- For $k = +1$, a statistically significant improvement for the cloned aux head over baseline.
- For $k = +2$, a highly significant degradation relative to baseline.

Qualitative inspection of predictions showed that:

- The cloned $t+1$ head often proposed plausible local continuations, sometimes matching the gold token more frequently than vanilla within the top-5.
- The $t+2$ head’s top-5 often contained syntactically reasonable tokens (articles, prepositions, punctuation) and occasionally the correct gold token, but overall confidence was low and ranking was inconsistent.

These findings suggest that **cloning the $t+1$ head helps preserve and slightly improve next-token behavior under the new loss**, but a randomly initialized $t+2$ head does not discover a strong, general direct mapping to future tokens when the trunk is frozen.

5.4 From-scratch Multi-Token Model on FineWeb (FutureGPT)

To probe the behavior of a small Meta-style model trained end-to-end, I built a two-head GPT-style model from scratch and trained it on a streaming subset of FineWeb:

- Shared decoder trunk with two linear heads for $t+1$ and $t+2$.
- Loss weights λ_1 and λ_2 adjusted over training to emphasize both offsets while stabilizing $t+1$.

After training to a fixed step, I evaluated on 819,200 tokens. Results were qualitatively:

- $t+1$: moderate loss and perplexity, with top-1 accuracy around the mid-20% range and top-5 around the mid-40% range.
- $t+2$: higher loss and perplexity, with top-1 accuracy in the low teens and top-5 under 30%.

Qualitative examples showed that:

- For some positions, the $t+2$ head predicted tokens that matched the gold token or plausible continuations (e.g., predicting “of” or commas where appropriate).
- More often, the $t+2$ head’s distribution was diffuse and dominated by frequent, generic tokens, while $t+1$ provided sharper predictions.

Overall, the model **did learn a non-trivial $t+2$ signal** from scratch, but it was much weaker than the $t+1$ signal and far from the accuracy of a standard next-token model at $k = +1$ on similar data.

5.5 Additional Analyses and Side Experiments

The notebook also includes:

- **Visualization of prediction patterns** across positions for fixed prompts, printing $t+1$ and $t+2$ predictions for each input token. These demonstrate that the $t+1$ head tracks local syntactic structure reasonably well, while $t+2$ predictions often revert to high-frequency tokens like “of”, “the”, and punctuation.
- **Representation geometry explorations** on LLaMA-2-7B:
 - A sentence-similarity heatmap built from normalized final-layer embeddings shows clear block structure: semantically similar excerpts cluster with high cosine similarity.

- A small “innate concept” probing experiment extracts random passages, embeds them with LLaMA-2-7B, and finds nearest token embeddings. The resulting nearest-neighbor tokens are often obscure or morphologically related, illustrating how model-specific embedding geometry may not align with human concept labels.
- **UMAP projections** of sentence-level embeddings to visualize clustering, confirming that different prompts occupy distinct regions in embedding space, consistent with recent work on intermediate-layer representations.

These side experiments are not central to the quantitative conclusions but provide qualitative context about how representations organize information.

6 Results

6.1 Summary of Quantitative Findings

Key patterns across experiments:

- **Baseline GPT-2 (WikiText-2, teacher-forced):**
 - Next-token and two-step-ahead prediction both show reasonably strong hits@k and moderate loss, indicating rich future-token information in hidden states.
- **Experiment 1 – Random aux heads (frozen trunk):**
 - The aux t+1 and t+2 heads learned to some extent but **did not reach baseline performance**, with t+2 especially weak.
- **Experiment 2 – Cloned t+1 head + random t+2 (frozen trunk):**
 - The cloned t+1 head **slightly improved** on the baseline GPT-2 head under a multi-offset loss.
 - The aux t+2 head remained substantially worse than baseline teacher-forced t+2 decoding, despite explicit supervision.
 - McNemar tests confirmed that these differences are statistically significant.
- **From-scratch FutureGPT MTP (FineWeb, 2-head):**
 - The t+1 head achieved a modest but usable accuracy and perplexity.
 - The t+2 head obtained non-trivial but clearly weaker performance, with substantially higher perplexity and lower top-k accuracy.

6.2 Interpretation

These results support several conclusions:

1. **Frozen-trunk auxiliary heads alone are insufficient to match or exceed baseline future-token performance.** Even with substantial training, the aux $t+2$ heads fall far below the teacher-forced baseline $t+2$ accuracy of GPT-2.
2. **Cloning the $t+1$ head can improve next-token performance under a multi-offset loss**, indicating that the shared trunk does have enough capacity and structure for a slightly better mapping when retrained in this configuration.
3. **Direct $t+2$ prediction remains difficult** for these small models, both with frozen trunks and in a from-scratch MTP model, suggesting that much of the future-token signal is indirect and sensitive to how the trunk itself is trained.
4. **Qualitative patterns show breadcrumb-like behavior.** Many cases where $t+2$ is predicted correctly seem to arise from the model exploiting normal autoregressive structure rather than learning a robust, dedicated $t+2$ mapping.

7 Discussion

Wu et al. argue that small GPT-2 models primarily rely on breadcrumbs rather than deliberate pre-caching. My experiments are consistent with this view. The frozen GPT-2 trunk clearly contains enough information for a high-performing teacher-forced $t+2$ predictor, as evidenced by the baseline results. However, when restricted to training only new linear heads, the model:

- Can slightly reshape the $t+1$ mapping and even outperform the original head when initialized from it.
- Struggles to produce a strong direct $t+2$ mapping, even when $t+2$ is explicitly supervised.

This suggests that much of the future-token predictability in GPT-2’s hidden states is **entangled** with the next-token objective. The trunk representations appear optimized to support autoregressive rollouts rather than separate direct mappings to x_{t+2} or x_{t+3} .

Meta’s MTP results demonstrate that multi-token heads can yield large gains in larger models when the trunk is trained under the MTP objective from the start. In that regime, off-diagonal gradient signals from multiple future positions act throughout training, encouraging the trunk to encode more decodable future structure. By contrast, my GPT-2 experiments hold the trunk fixed, which effectively removes this pathway: auxiliary heads must work with a representation that was never optimized for direct multi-token decoding.

The from-scratch FutureGPT experiment partially bridges this gap. Training a small model end-to-end with two heads does produce a meaningful $t+2$ signal, but it remains much weaker than $t+1$. This aligns with Meta’s observation that MTP benefits are not uniform across scales or tasks, and with Wu’s finding that pre-caching is limited in smaller models.

The side experiments on representation geometry and innate concepts in LLaMA-2 embeddings further reinforce that **representation quality varies across layers and tasks**. While my work focuses on the final layer of GPT-style models, recent work suggests that intermediate layers might encode richer, more linearly decodable structure. A natural next step would be to attach auxiliary heads or probes at intermediate layers rather than only at the top.

8 Limitations

This study has several important limitations:

- **Model scale:** All GPT-style experiments use GPT-2 small or smaller custom models. Results may not generalize to larger models where Wu et al. find stronger pre-caching and Meta reports greater MTP gains.
- **Frozen trunk assumption:** The main auxiliary-head experiments keep the GPT-2 trunk entirely frozen. This isolates the effect of heads but prevents the representation from adapting to the multi-token objective, likely underestimating what full MTP training could achieve.
- **Limited datasets:** GPT-2 experiments are confined to WikiText-2, and the MTP model is trained only on a streaming slice of FineWeb. No large-scale benchmark (e.g., MMLU or code datasets) is included.
- **Evaluation scope:** While I compute hits@k and apply McNemar tests for top-1 correctness, I do not perform a full layerwise probe analysis across all layers, nor do I explore the impact of architectural changes beyond adding linear heads.
- **No direct comparison to a full-scale MTP implementation:** My from-scratch MTP model is substantially smaller and trained for fewer steps than those in Meta’s work, so results should be interpreted as a small-scale proof-of-concept rather than a direct reproduction.

9 Future Work

Building on this study, natural extensions include:

1. **Unfreezing the trunk:** Train GPT-2 (or a similar decoder) end-to-end under a multi-token objective, comparing performance and representations to both the frozen-head and vanilla baselines.
2. **Intermediate-layer probes and heads:** Attach heads or linear probes at multiple layers to test whether mid-layer representations support stronger future-token prediction, connecting directly to work on layerwise representation quality.
3. **Scaling up:** Repeat these experiments with larger models (e.g., Pythia, LLaMA variants) to examine how pre-caching and MTP benefits scale with parameter count, echoing Wu’s results on myopia gaps.
4. **Synthetic tasks for controlled pre-caching:** Design synthetic sequence tasks similar to Wu’s constructions but tailored for multi-token heads, to measure how quickly and robustly models learn necessary pre-caching.
5. **Speculative decoding experiments:** Use trained multi-token heads to drive speculative decoding and quantify speed–quality tradeoffs, comparing against standard speculative decoding with a separate draft model.
6. **Mechanistic interpretability:** Investigate specific attention heads and MLP neurons responsible for any observed future-token behavior using feature visualization and intervention techniques.

10 Conclusion

This independent study documents a series of experiments on future-token prediction in small GPT-style models. By implementing transformers from scratch, rearchitecting GPT-2 with auxiliary heads, and training a small Meta-style MTP model, I systematically tested how much future-token structure can be extracted from existing hidden states and what role auxiliary heads play.

The main findings are:

- Frozen-trunk auxiliary heads can slightly improve next-token performance when initialized from the original head but **cannot** match baseline performance for direct $t+2$ prediction.
- A small from-scratch MTP model learns a non-trivial $t+2$ predictor but remains much weaker than $t+1$, with high perplexity and moderate top-5 accuracy.
- These observations are consistent with Wu et al.’s conclusion that small models primarily lay down breadcrumbs rather than heavily pre-caching for the future, and with Meta’s result that MTP’s strongest benefits appear at larger scales and with full-trunk training.

Overall, the experiments support a nuanced view: **small GPT-style models do encode some future-token information in their hidden states, but exploiting it effectively appears to require training the trunk under multi-token objectives or scaling up models, rather than simply adding auxiliary heads to a frozen network.**

11 References

- Wu, W., Morris, J. X., & Levine, L. (2024). *Do Language Models Plan Ahead for Future Tokens?* COLM 2024.
- Gloeckle, F., et al. (2024). *Better & Faster Large Language Models via Multi-Token Prediction.*
- Skean, O., et al. (2025). *Layer by Layer: Uncovering Hidden Representations in Language Models.*
- Vaswani, A., et al. (2017). *Attention is All You Need.*
- Radford, A., et al. (2019). *Language Models are Unsupervised Multitask Learners.*